

Fundamental Level (Laboratorios)

Modulo. Entorno y Herramientas:

- Crear proyecto con Poetry, activar venv, instalar black/isort/ruff
- Configurar pre-commit; corregir infracciones PEP 8 iniciales

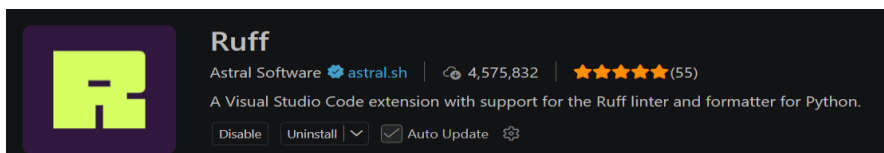
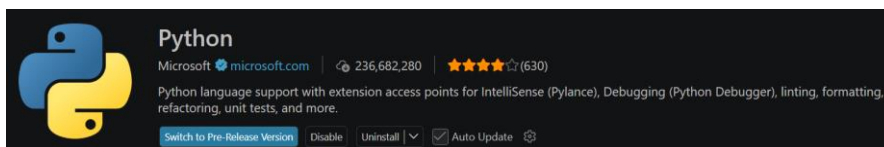
1. Instalación de Python

Instalamos la versión más estable de Python en nuestro sistema y nos aseguramos de que esté disponible globalmente:

```
C:\Users\juan.rivera>py --version  
Python 3.13.15
```

2. Configuración del IDE (VS Code)

Abrimos VS Code e instalamos las extensiones básicas en mi caso Python (de Microsoft) y Ruff (de Astral).



3. Gestión de proyectos y entorno con Poetry

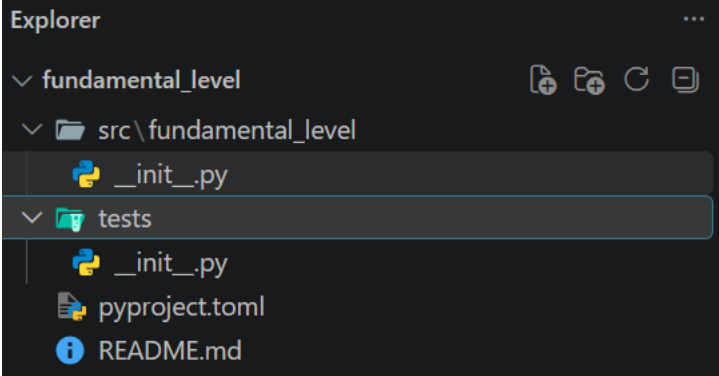
Nos olvidamos temporalmente de venv manual y requirements.txt, ya que Poetry se encarga de crear el entorno virtual y el archivo pyproject.toml al mismo tiempo.

Verificamos que Poetry se haya instalado correctamente:

```
C:\Users\juan.rivera>poetry --version  
Poetry (version 2.4.3)
```

Creamos nuestro proyecto con el comando “poetry new nombredelproyecto”:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python
$ poetry new fundamental_level
Created package fundamental_level in fundamental_level
```



The Explorer sidebar in VS Code shows the following structure:

- fundamental_level
 - src\fundamental_level
 - __init__.py
 - tests
 - __init__.py
 - pyproject.toml
 - README.md

4. Formato y Calidad de Código (PEP 8 + Ruff)

Agregamos Ruff como dependencia de desarrollo en el proyecto con “poetry add –group dev ruff”

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level
$ poetry add --group dev ruff
Using version ^0.16.7 for ruff

Updating dependencies
Resolving dependencies... (0.1s)

No dependencies to install or update

Writing lock file
```

Y ejecutamos “poetry run ruff check .” para analizar nuestro código:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level
$ poetry run ruff check .
All checks passed!
```

5. Automatización con Pre-commit

Aseguramos que ningún código sin formatear o con errores suba a nuestro repositorio:

Agregamos pre-commit a las dependencias de desarrollo con “poetry add –group dev pre-commit”

```

juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/fundamental_level
$ poetry add --group dev pre-commit
Using version ^4.6.2 for pre-commit

Updating dependencies
Resolving dependencies... (0.3s)

No dependencies to install or update

Writing lock file

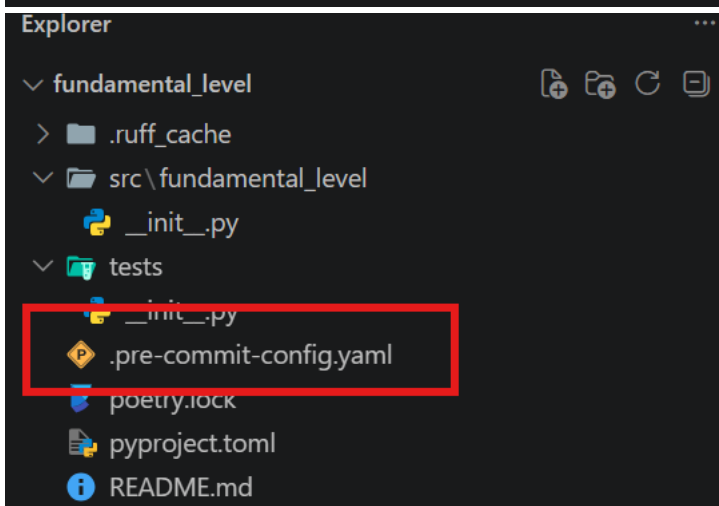
```

Creamos el archivo “.pre-comit-config.yaml” para indicarle que ejecute Ruff antes de cada git commit.

```

juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/fundamental_level
$ touch .pre-commit-config.yaml

```



Instalamos los hooks ejecutando “poetry run pre-commit install”:

```

juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/fundamental_level
$ git init
Initialized empty Git repository in C:/Users/juan.rivera/Documents/Curso_Python/fundamental_level/.git/

juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/fundamental_level (master)
$ poetry run pre-commit install
pre-commit installed at .git\hooks\pre-commit

```

Primer commit automatizado:

Abrimos el archivo principal dentro de nuestro paquete y escribimos código con errores intencionales y variables no usadas:

```
.gitignore U X  __init__.py 3, U ●
src > fundamental_level > __init__.py > ...
1  import os
2  import sys
3
4  def Saludar_Usuario(nombre):
5      mensaje = f"Hola, {nombre}"
6      print( mensaje )
7
8  saludo = "Hola mundo"
```

Y al tratar de hacer el commit el pre-commit intercepta el commit automáticamente antes de guardar los cambios, corre Ruff Linter y Ruff Formatter.

En la terminal vemos cómo Ruff formatea los espacios innecesarios y nos advierte sobre las librerías os y sys importadas, pero no utilizadas.

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (master)
$ git commit -m "feat: Módulo. Entorno y Herramientas"
[INFO] Initializing environment for https://github.com/astral-sh/ruff-pre-commit.
[INFO] Installing environment for https://github.com/astral-sh/ruff-pre-commit.
[INFO] Once installed this environment will be reused.
[INFO] This may take a few minutes...
ruff.....Failed
- hook id: ruff
- exit code: 1

src\fundamental_level\__init__.py:1:8: F401 `os` imported but unused
|
1 | import os
|      ^^ F401
2 | import sys
|
= help: Remove unused import: `os`

src\fundamental_level\__init__.py:2:8: F401 `sys` imported but unused
|
1 | import os
2 | import sys
|      ^^^ F401
|
= help: Remove unused import: `sys`

Found 2 errors.

ruff-format.....Failed
- hook id: ruff-format
```

Corregimos

nuestro

código:

```
def Saludar_Usuario(nombre):  
    mensaje = f"Hola, {nombre}"  
    print(mensaje)  
  
saludo = "Hola mundo"
```

Y realizamos el commit:

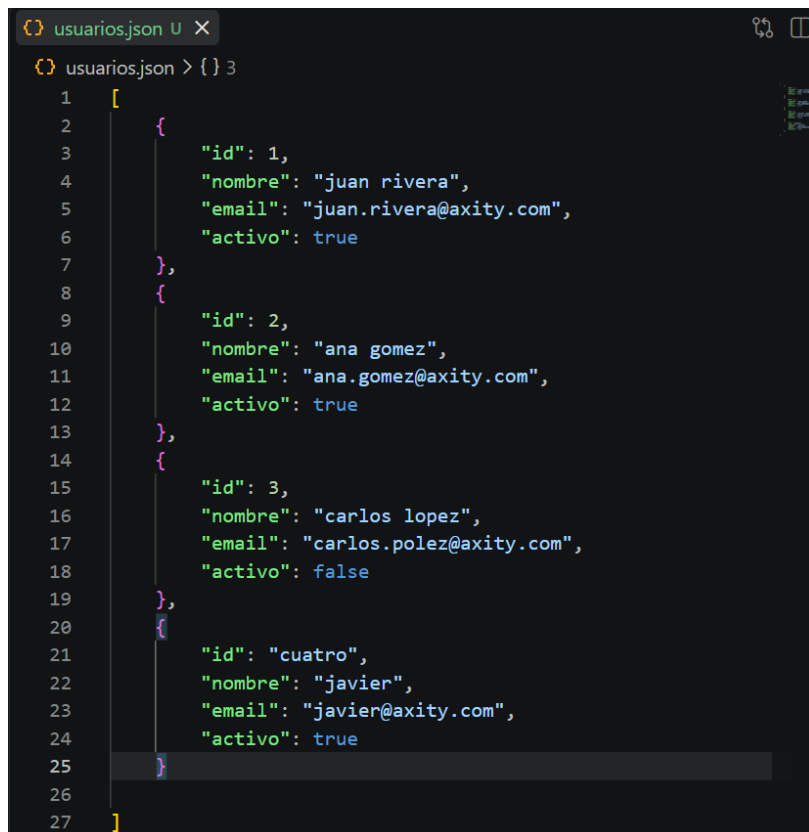
```
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/fundamental_level (master)  
$ git add .  
  
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/fundamental_level (master)  
$ git commit -m "feat: Módulo. Entorno y Herramientas"  
ruff.....Passed  
ruff-format.....Passed  
[master (root-commit) bcaaaff] feat: Módulo. Entorno y Herramientas  
7 files changed, 286 insertions(+)  
create mode 100644 .gitignore  
create mode 100644 .pre-commit-config.yaml  
create mode 100644 README.md  
create mode 100644 poetry.lock  
create mode 100644 pyproject.toml  
create mode 100644 src/fundamental_level/__init__.py  
create mode 100644 tests/__init__.py
```

Modulo. Fundamentos del Lenguaje:

- Script que lee JSON, filtra/agrega datos y maneja errores de archivo/formato

1. Crear usuarios .json en la raíz:

Creemos el archivo usuarios.json en la raíz de nuestro proyecto, este contiene registros intencionalmente válidos e inválidos para poner a prueba nuestros filtros y el manejo de errores:



```
1  [
2      {
3          "id": 1,
4          "nombre": "juan rivera",
5          "email": "juan.rivera@axity.com",
6          "activo": true
7      },
8      {
9          "id": 2,
10         "nombre": "ana gomez",
11         "email": "ana.gomez@axity.com",
12         "activo": true
13     },
14     {
15         "id": 3,
16         "nombre": "carlos lopez",
17         "email": "carlos.polez@axity.com",
18         "activo": false
19     },
20     {
21         "id": "cuatro",
22         "nombre": "javier",
23         "email": "javier@axity.com",
24         "activo": true
25     }
26 ]
27 ]
```

2. Implementar src/fundamental_level/main.py

Construimos el script principal, cuyo objetivo es actuar como filtro de calidad de datos automatizados y seguro:

```
import json
import re
from pathlib import Path

def procesar_usuarios(ruta_relativa: str) -> list[dict]:
    path = Path(ruta_relativa)
```

```

patron_email = r"^[\w\.-]+@[\w\.-]+\.\w+$"
usuarios_validos = []

try:
    with open(path, "r", encoding="utf-8") as archivo:
        datos = json.load(archivo)

    for registro in datos:
        # Pattern Matching (Python 3.10+): Valida llaves, tipos y
estado
        match registro:
            case {
                "id": int(user_id),
                "nombre": str(nombre),
                "email": str(email),
                "activo": True,
            }:
                # Expresión Regular: Validar sintaxis del email
                if re.match(patron_email, email):
                    usuarios_validos.append(
                        {
                            "id": user_id,
                            "nombre": nombre.strip().title(),
                            "email": email,
                        }
                    )
            case _:
                # Omite registros inactivos, con id no entero o
incompletos
                continue

except FileNotFoundError:
    print(f"Error: No se encontró el archivo '{ruta_relativa}'.")
except json.JSONDecodeError:
    print(f"Error: El archivo '{ruta_relativa}' no contiene un JSON
válido.")
except Exception as e:
    print(f"Error inesperado al procesar el archivo: {e}")

return usuarios_validos

```

```
if __name__ == "__main__":
    resultado = procesar_usuarios("usuarios.json")
    print(f"Usuarios procesados con éxito ({len(resultado)}):")
    print(resultado)
```

“patron_email” sirve para verificar que la cadena del correo cumpla con el formato estándar de un email:

```
patron_email = r"^[\\w\\.\\-]+@[\\w\\.\\-]+\\.\\w+$"
```

- ^: Indica el inicio del texto
- [\\w\\.\\-]+: Acepta una o más letra, números, guiones bajos (\\w), puntos (.) o guiones (-).
- @: Exige la presencia obligatoria del carácter @.
- [\\w\\.\\-]+: Acepta el nombre del dominio.
- \\w+\$: Exige un punto seguido de la extensión hasta el final de la cadena (\$).

Manejo seguro del archivo y control de excepciones:

```
try:
    with open(path, "r", encoding="utf-8") as archivo:
        datos = json.load(archivo)
    ...
except FileNotFoundError:
    ...
except json.JSONDecodeError:
    ...
```

- with open(...): Abre el archivo de forma segura y garantiza que se cierre automáticamente al terminar.
- json.load(archivo): CONvierte el contenido del archivo JSON en una lista de diccionarios nativa de Python.
- except FileNotFoundError: Si el archivo .json no existe en la ruta especificada, en lugar de romper el programa con un error en pantalla (crash), atrapa la excepción y muestra un mensaje amigable.
- except json.JSONDecodeError: Si el archivo JSON tiene un error sintáctico, atrapa el fallo e informa que el formato es inválido.

Filtrado Estructural con Pattern Matching:


```

for registro in datos:
    match registro:
        case {
            "id": int(user_id),
            "nombre": str(nombre),
            "email": str(email),
            "activo": True,
        }:

```

Aquí se evalúa cada elemento de la lista JSON. Se validan tres cosas al mismo tiempo:

- Estructura: Que el diccionario contenga exactamente las llaves “id”, “nombre”, “email” y “activo”.
- Tipos de datos: Que “id” sea un número entero, que “nombre” sea una cadena de texto y que “email” igual sea una cadena de texto.
- Estado: Que la llave “activo” sea explícitamente True.

```

if re.match(patron_email, email):
    usuarios_validos.append(
        {
            "id": user_id,
            "nombre": nombre.strip().title(),
            "email": email,
        }
    )

```

Si el registro pasó el filtro:

- Se evalúa si el correo es válido con `re.match(patron_email, email)`.
- Si el correo cumple la regla, limpiamos el texto del nombre quitando espacios extras (`.strip()`) y convirtiendo en mayúsculas las primeras letras (`.title()`), convirtiendo “juan rivera” en “Juan Rivera”.
- Se guardan únicamente los registros limpios en la lista `usuarios_validos`.

3. Ejecutar el script:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ poetry run python src/fundamental_level/main.py
Usuarios procesados con éxito (2):
[{'id': 1, 'nombre': 'Juan Rivera', 'email': 'juan.rivera@axity.com'},
{'id': 2, 'nombre': 'Ana Gomez', 'email': 'ana.gomez@axity.com'}]
```

Verificamos que cumpla con PEP 8:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ poetry run ruff check src/fundamental_level/main.py --fix
All checks passed!
```

Realizamos nuestro commit y push:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git add .

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git commit -m "feat: Módulo. Fundamentos de Lenguaje"
ruff.....Passed
ruff-format.....Passed
[main 1142f60] feat: Módulo. Fundamentos de Lenguaje
 2 files changed, 76 insertions(+)
 create mode 100644 usuarios.json

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git push origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (11/11), 3.54 KiB | 1.77 MiB/s, done.
Total 11 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/juanrivera-A/capacitacion-python.git
 bcaaaff..1142f60 main -> main
```

Modulo 3. Funciones y programación “pythonic”

1. Decorador con argumentos (`retry_ with_backoff`): Con esto se reintenta la ejecución de una función si ocurre un error, incrementando el tiempo de espera exponencialmente entre intentos.

```
def reintento_con_backoff(
    max_reintentos: int = 3, tiempo_base: float = 1.0
) -> Callable:
    """ Decorador con argumentos que reintenta la ejecución tras una
    excepción.

    Aplica backoff exponencial: espera base * (2 ** intento) segundos.
    """

    def decorador(func: Callable) -> Callable:
        @wraps(func)
        def wrapper(*args, **kwargs):
            for intento in range(max_reintentos):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    espera = tiempo_base * (2**intento)
                    print(
                        f"Error en '{func.__name__}': {e}. Reintento
{intento + 1}/{max_reintentos} en {espera:.1f}s..."
                    )
                    time.sleep(espera)

            # Último intento para lanzar la excepción si falla
            definitivamente
            print(
                f"Excedio el número máximo de reintentos en
'{func.__name__}.'"
            )
            return func(*args, **kwargs)

        return wrapper

    return decorador
```

2. Generador por lotes (batch_generator): Procesa una secuencia de datos dividiendola en bloques (chunks) para minimizar el consumo de memoria RAM.

```
def generador_por_lotes(datos: list[dict], tamaño_lote: int) -> Generator[list[dict]]:
    """Procesa sublistas de datos de un tamaño fijo para evitar cargas pesadas en memoria."""
    for i in range(0, len(datos), tamaño_lote):
        yield datos[i : i + tamaño_lote]
```

3. Context Manager (TimerContext): Mide con precisión el tiempo que tarda un bloque de código ejecutado dentro de un bloque whit.

```
@contextmanager
def temporizador(etiqueta: str):
    "Context manager para medir el tiempo transcurrido en un bloque ``whit.``"
    inicio = time.perf_counter()
    print(f"[Inicio: {etiqueta}]")
    try:
        yield
    finally:
        fin = time.perf_counter()
        print(f"[Fin: {etiqueta}] Tiempo total: {fin - inicio:.4f} segundos\n")
```

Modulo 4. Objetos y Modelados de Datos

1. Definir las entidades de dominio (dataclass): En este primer paso se crea la entidad inmutable OrderItem y la entidad principal Order usando @dataclass

Requerimiento de Dominio:

1. OrderItem: *productid*, *quantity* y *unitprice*. Incluye la propiedad calculada *subtotal*.
2. Order: *customerid*, *items*, *orderid* y *created_at*.
3. Cálculo Derivado: Propiedad @property *total_amount* que suma el subtotal de todos los items.
4. Comparaciones (Dunder Methods): Método `__lt__` para comparar dos pedidos por su *total_amount* usando `<`.

#1. Entidades de Dominio (DATACLASSES)

```
@dataclass(frozen=True)
class OrderItem:
    """ Item individual de un pedido. Inmutable (frozen)."""

    product_id: str
    quantity: int
    unit_price: Decimal

    @property
    def subtotal(self) -> Decimal:
        """ Cálculo derivado del subtotal de esta línea """
        return self.quantity * self.unit_price


@dataclass
class Order:
    """ Entidad principal de Pedido

    Permite cálculo de total y comparación por monto usando dunder methods
    """

    customer_id: str
    items: list[OrderItem] = field(default_factory=list)
    order_id: UUID = field(default_factory=uuid4)
    created_at: datetime = field(default_factory=datetime.now)

    @property
    def total_amount(self) -> Decimal:
        """Cálculo derivado del monto total del pedido."""
        return sum((item.subtotal for item in self.items),
Decimal("0.00"))

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Order):
            return NotImplemented
        return self.order_id == other.order_id

    def __lt__(self, other: "Order") -> bool:
```

```

    """Permite comparar pedidos por monto total (ordemaniento)."""
    if not isinstance(other, Order):
        return NotImplemented
    return self.total_amount < other.total_amount

```

2. La definición de los esquemas de Pydantic (OrderIn y OrderOut)

Se define:

- OrderItemSchema: Validación individual de los productos (cantidades mayores a 0, precios positivos).
- OrderIn: Esquema de entrada con un validador personalizado (@field_validator) para asegurar que la lista de ítems no esté vacía.
- OrderOut: Esquema de salida serializable a JSON con los totales ya calculados.

```

# 2. Esquemas de validación y serialización (PYDANTIC)

class OrderItemSchema(BaseModel):
    """Esquema de entrada/salida para un ítem del pedido."""

    product_id: str = Field(..., min_length=3, description = "SKU o ID del producto")
    quantity: int = Field(..., gt=0, description = "La cantidad debe ser mayor a 0")
    unit_price: Decimal = Field(..., gt=0, description = "El precio unitario debe ser mayor a 0")

class OrderIn(BaseModel):
    """Modelo DTO para recibir la petición de creación de un pedido."""

    customer_id: str = Field(..., min_length=1)
    items: list[OrderItemSchema] = Field(..., min_length=1)

    @field_validator("items")
    @classmethod
    def validar_items_no_vacios(cls, items: list[OrderItemSchema]) -> list[OrderItemSchema]:
        """Asegura que la orden contenga al menos un ítem válido."""
        if not items:
            raise ValueError("El pedido debe contener al menos un elemento.")
        return items

```

```

class OrderOut(BaseModel):
    """Modelo DTO de salida serializable a JSON."""

    Order_id: UUID
    customer_id: str
    items: list[OrderItemSchema]
    total_amount: Decimal
    created_at: datetime

```

3. Escribimos las funciones de conversión entre la capa de entrada/salida(DTOs de Pydantic) y nuestro dominio (Order dataclass), y ejecutar la prueba completa dentro del bloque if __name__ == "__main__":.

```

# 3. Funciones de conversión y flujo principal

def payload_a_entidad(payload: dict) -> Order:
    """Valida los datos de entrada mediante Pydantic y los convierte a
    la entidad Order."""
    # 1. Validación de entrada (Pydantic)
    order_in = OrderIn(**payload)

    # 2. Conversión a objetos de dominio (Dataclass)
    domain_items = [
        OrderItem(
            product_id=item.product_id,
            quantity=item.quantity,
            unit_price=item.unit_price,
        )
        for item in order_in.items
    ]

    return Order(

        customer_id=order_in.customer_id,
        items=domain_items,
    )

def entidad_a_dto_salida(order: Order) -> OrderOut:

```

```

    """Convierte la entidad interna Order al esquema de salida
    OrderOut."""
    return OrderOut(
        order_id=order.order_id,
        customer_id=order.customer_id,
        items=[
            OrderItemSchema(
                product_id=item.product_id,
                quantity=item.quantity,
                unit_price=item.unit_price,
            )
            for item in order.items
        ],
        total_amount=order.total_amount,
        created_at=order.created_at,
    )

```

4. Ejecución y Formateo con Ruff

- Ejecutamos el laboratorio:


```
$ poetry run python main.py
LABORATORIO MÓDULO 4: OBJETOS Y MODELOS DE DATOS

Pedido 1 Creado: IDa53d2f13-0294-4805-911e-ea7b80a7ce2c
Total calculado en la dataclass: $800.00

Pedido 2 Creado: Total $1200.00
El pedido 1 ($800.00) es MENOR que el Pedido 2 ($1200.00)

JSON de Salida (OrderOut):
{
  "order_id": "a53d2f13-0294-4805-911e-ea7b80a7ce2c",
  "customer_id": "CUST-9982",
  "items": [
    {
      "product_id": "PROD-A",
      "quantity": 2,
      "unit_price": "150.50"
    },
    {
      "product_id": "PROD-B",
      "quantity": 1,
      "unit_price": "499.00"
    }
  ],
  "total_amount": "800.00",
  "created_at": "2026-09-18T14:38:22.425275"
}
```

```
Probando entrada enválida (precio negativo y cantidad en 0)...
Pydantic capturó el error correctamente:
3 validation errors for OrderIn
customer_id
  Field required [type=missing, input_value={'customer_id': 'CUST-000', 'unit_price': '-50.00'}], input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
items.0.quantity
  Input should be greater than 0 [type=greater_than, input_value=0, input_type=int]
  For further information visit https://errors.pydantic.dev/2.13/v/greater_than
items.0.unit_price
  Input should be greater than 0 [type=greater_than, input_value='-50.00', input_type=str]
  For further information visit https://errors.pydantic.dev/2.13/v/greater_than
```

- Verificamos formato y linter con ruff:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundament
al_level/src/modulo4 (main)
$ poetry run ruff check main.py
All checks passed!
```

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundament
al_level/src/modulo4 (main)
$ poetry run ruff format main.py
1 file reformatted
```

- Realizamos el commit:

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundament
al_level (main)
$ git commit -m "feat: Módulo. Objetos y Modelos de Datos"
ruff.....
..Passed
ruff-format.....
..Passed
[main b26a8fb] feat: Módulo. Objetos y Modelos de Datos
8 files changed, 456 insertions(+), 1 deletion(-)
rename src/{pythonic => modulo3}/__init__.py (100%)
rename src/{pythonic => modulo3}/main.py (100%)
rename src/{pythonic => modulo3}/practica_teoría.py (100%)
create mode 100644 src/modulo4/__init__.py
create mode 100644 src/modulo4/main.py
create mode 100644 src/modulo4/practica_teoría.py
```

Modulo 5. Tipado Estático Opcional y Calidad

Objetivo del laboratorio:

Anotar tipos en el código previo, ejecutar mypy y ruff; configurar pre-commit

1. Desarrollar nuestro código con las anotaciones de tipo correspondientes (TypedDict, Protocol, Literal, Union, y uso documentado de # type: ignore).

```
from typing import Literal, Protocol, TypedDict

# 1. Typeddict y Literales (Estructuras y Estados)

class UsuarioResponseDict(TypedDict):
```

```
"""Estructura esperada al recibir un usuario de una API externa"""
```

```
id: int
username: str
rol: Literal["ADMIN", "USER", "GUEST"]
activo: bool
```

```
# 2. Protocol con duck typing estático
```

```
class RepositorioUsuario(Protocol):
    """Protocolo que define las operaciones de persistencia/consulta."""

    def obtener_por_id(sel, usuario_id: int) -> UsuarioResponseDict |
None: ...

    def guardar(self, usuario: UsuarioResponseDict) -> bool: ...
```

```
# 3. Implementación de repositorio (Sin herencia explicita)
```

```
class MemoriaUsuarioRepository:
    """Implementa RepositorioUsuario implícitamente por su firma."""

    def __init__(self) -> None:
        self._db: dict[int, UsuarioResponseDict] = {}

    def obtener_por_id(self, usuario_id: int) -> UsuarioResponseDict |
None:
        return self._db.get(usuario_id)

    def guardar(self, usuario: UsuarioResponseDict) -> bool:
        self._db[usuario["id"]] = usuario
        return True
```

```
# 4. Servicio de negocio y manejo de tipo opcional
```

```

class ServicioUsuario:
    def __init__(self, repo: RepositorioUsuario) -> None:
        self.repo = repo

    def registrar_usuario(
        self, usuario_id: int, username: str, rol: Literal["ADMIN",
"USER", "GUEST"]
    ) -> bool:
        nuevo_usuario: UsuarioResponseDict = {
            "id": usuario_id,
            "username": username,
            "rol": rol,
            "activo": True,
        }
        return self.repo.guardar(nuevo_usuario)

    def obtener_rol_normalizado(self, usuario_id: int) -> str:
        usuario = self.repo.obtener_por_id(usuario_id)

        # Manejo estricto de Opcional / None exigido pro mypy
        if usuario is None:
            return "DESCONOCIDO"
        return usuario["rol"].lower()

if __name__ == "__main__":
    print("LABORATORIO MÓDULO 5: TIPADO ESTATICO Y CALIDAD")

    repo = MemoriaUsuarioRepository()
    servicio = ServicioUsuario(repo)

    # 1. Registro y consulta exitosos
    servicio.registrar_usuario(1, "jrivera", "ADMIN")
    rol = servicio.obtener_rol_normalizado(1)
    print(f"Usuario registrado correctamente. Rol normalizado: {rol}")

    # 2. Desmostración de dinamismo vs. mypy con type: ignore
    payload_dinamico: dict = {
        "id": 2,
        "username": "guest_user",
        "rol": "INVITADO",
        "activo": True,
    }

```

```

    }

    # Documentamos la excepción explícitamente por que "INVITADO" no
    pertenece al Literal
    repo.guardar(payload_dinamico) # type: ignore[arg-type]
    print("Obejto guardado forzando el tipo mediante bypass
    documentado.")

```

2. Ejecutamos las herramientas de analisis (mypy y ruff):

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundament
al_level/src/modulo5 (main)
$ poetry run mypy main.py
Success: no issues found in 1 source file

```

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundament
al_level/src/modulo5 (main)
• $ poetry run ruff check main.py
All checks passed!

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundament
al_level/src/modulo5 (main)
• $ poetry run ruff format main.py
1 file left unchanged

```

3. Configurar Pr-commit y CI Checks

- Nos aseguramos de tener pre-commit instalado como dependencia de desarrollo:

```

$ poetry add --group dev pre-commit
The following packages are already present in the pyproject.toml and will be skipped:

- pre-commit

If you want to update it to the latest compatible version, you can use `poetry update package`.
If you prefer to upgrade it to the latest available version, you can use `poetry add package@latest`.

Nothing to add.

```

- Actualizamos el archivo .pre-commit-config.yaml:

```

P .pre-commit-config.yaml
1  repos:
2    - repo: https://github.com/astral-sh/ruff-pre-commit
3      rev: v0.6.0
4      hooks:
5        # Ejecuta el linter de Ruff para encontrar errores
6        - id: ruff
7          args: [--fix]
8        # Ejecuta el formateador de Ruff para mantener el estilo PEP 8
9        - id: ruff-format
10
11   - repo: https://github.com/pre-commit/mirrors-mypy
12     rev: v1.11.0
13     hooks:
14       - id: mypy
15       additional_dependencies: []

```

- Instalamos los hooks en nuestro entorno de Git:

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ poetry run pre-commit install
pre-commit installed at .git\hooks\pre-commit

```

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ poetry add "pydantic>=2.0"

Updating dependencies
Resolving dependencies... (1.5s)

Package operations: 5 installs, 0 updates, 0 removals

- Installing typing-extensions (4.16.0)
- Installing annotated-types (0.8.0)
- Installing pydantic-core (2.46.5)
- Installing typing-inspection (0.4.4)
- Installing pydantic (2.13.5)

Writing lock file

```

4. Se valida la automatización y hacer Commit:

```

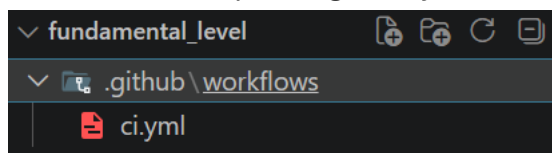
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git add .

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git commit -m "feat: Módulo. Tipado estático opcional y calidad"
ruff.....Passed
ruff-format.....Passed
mypy.....Passed
[main 4c0ffe4] feat: Módulo. Tipado estático opcional y calidad
7 files changed, 538 insertions(+), 4 deletions(-)
create mode 100644 .github/workflows/ci.yml
create mode 100644 src/modulo5/__init__.py
create mode 100644 src/modulo5/main.py
create mode 100644 src/modulo5/practica_teoría.py

```

5. Crear Workflow de CI en GitHub Actions:

- Para dejar la integración completa en el servidor, se crea un archivo de flujo de trabajo (workflow) para GitHub Actions.
- Creamos las carpetas .github y workflow, y el archivo ci.yml



- Agregamos la configuración de pipeline en .github/workflows/ci.yml

```

name: CI Quality Checks

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  quality-checks:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout del código
        uses: actions/checkout@v4

```

- name: Configurar Python
 - uses: actions/setup-python@v5
 - with:
 - python-version: '3.12'
- name: Instalar Poetry
 - run: pipx install poetry
- name: Instalar dependencias
 - run: poetry install --no-root
- name: Ejecutar Ruff Linter
 - run: poetry run ruff check .
- name: Ejecutar Ruff Formatter
 - run: poetry run ruff format --check .
- name: Ejecutar Mypy (Type Checking)
 - run: poetry run mypy src/

- Realizamos el push:

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git push origin main
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (12/12), 15.96 KiB | 3.19 MiB/s, done.
Total 12 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/juanrivera-A/capacitacion-python.git
   b26a8fb..4c0ffe4  main -> main

```


- Errores

presentados:



CI Quality Checks: All jobs have failed

View workflow run

Status	Job	Annotations
 quality-checks	CI Quality Checks / quality-checks Failed in 13 seconds	 annotations for CI Quality Checks / quality-checks 3

← Controles de calidad de CI

 **hazña: Módulo. Tipado estático opcional y calidad #1**



Resumen ▾

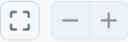
Activado mediante pulsación Hace 3 días

 Juan Rivera-A empujado  4 café 4 principal

Estado	Duración total	Artefactos
Falla	<u>19s</u>	—

ci.yml
encendido: pulsar

 controles de calidad 13 segundos



✖ Instalar dependencias 1s

```
1 ▶ Run poetry install --no-root
11 The currently activated Python version 3.12.14 is not supported by the project
    (>=3.13).
12 Trying to find and use a compatible version.
13
14 Poetry was unable to find a compatible version. If you have one, you can
    explicitly use it via the "env use" command.
15 Error: Process completed with exit code 1.
```

- Corrección de error (The currently activated Python version 3.12.14 is not supported by the project (>=3.13):

Nuestro proyecto requiere Python 3.13 o superior, pero en el workflow de GitHub Actions de GitHub Actions se colocó la versión 3.12:

```
steps:
  - name: Checkout del código
    uses: actions/checkout@v4

  - name: Configurar Python
    uses: actions/setup-python@v5
    with:
      python-version: '3.13'
```

- Se guarda el archivo y se hace commit/push:

```
juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git commit -m "fix(ci): Actualización de la versión de Python a 3.13 en el flujo de trabajo."
ruff.....(no files to check)Skipped
ruff-format.....(no files to check)Skipped
mypy.....(no files to check)Skipped
[main 578020a] fix(ci): Actualización de la versión de Python a 3.13 en el flujo de trabajo.
 1 file changed, 1 insertion(+), 1 deletion(-)

juan.rivera@mxjuriagdell15400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 450 bytes | 450.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/juanrivera-A/capacitacion-python.git
 4c0ffe4..578020a  main -> main
```

- Verificamos en GitHub:

Triggered via push 6 minutes ago

 **juanrivera-A** pushed  578020a main

Status	Total duration	Artifacts
Failure	<u>33s</u>	—

ci.yml

on: push

 **quality-checks** 28s

- Nuevos errores encontrados:

▼  **Ejecutar Mypy (Type Checking)** 4s

```
1  ► Run poetry run mypy src/
11 src/modulo4/practica_teoría.py:63: error: Argument 1 to "CursoIn" has
   incompatible type "**dict[str, Sequence[Collection[str]]]"; expected "str"
   [arg-type]
12 src/modulo4/practica_teoría.py:63: error: Argument 1 to "CursoIn" has
   incompatible type "**dict[str, Sequence[Collection[str]]]"; expected
   "list[EstudianteIn]" [arg-type]
13 src/modulo4/practica_teoría.py:77: error: Item "None" of "Estudiante | None"
   has no attribute "nombre" [union-attr]
14 src/modulo4/practica_teoría.py:77: error: Item "None" of "Estudiante | None"
   has no attribute "calificacion_promedio" [union-attr]
15 Found 4 errors in 1 file (checked 12 source files)
16 Error: Process completed with exit code 1.
```

El pipeline falló en el paso de mypy debido a errores en un archivo del módulo anterior:
src/modulo4/practica_teoría.py.

Al ejecutar poetry run mypy src/ en GitHub Actions, la herramienta ecanéa todo el directorio src/ , detectando observaciones en el modulo4.

Diagnóstico de los errores:

1. Línea 63 (arg.type): Se está desempaquetando un diccionario (**dict) hacia CursorIn, pero mypy no puede inferir los tipos correctos para str y list[EstudianteIn].
2. Línea 77(union-attr): Se intenta acceder a .nombre y .calificacion_promedio sobre una variable de tipo Estudiante | None sin validar primero si es None.

Solucion:

1. Corregir el manejo de None en la línea 77
Nos aseguramos de agregar una verificación if antes de acceder a las propiedades del estudiante.
2. Corregir la instanciación de CursorIn en la línea 63

En lugar de desempaquetar directamente un diccionario genérico (**payload), pasamos los argumentos de forma explícita.

Fragmento de código corregido:

```
__name__ == "__main__":
    print("PRÁCTICA TEÓRICA: MÓDULO 4\n")

    raw_payload = {
        "nombre_curso": "Python Backend Avanzado",
        "estudiantes": [
            {
                "matricula": "EST-01",
                "nombre": "Juan",
                "calificacion_promedio": "8.5",
            },
            {
```

```

        "matricula": "EST-02",
        "nombre": "María",
        "calificacion_promedio": "9.8",
    },
],
}

# Validar entrada con Pydantic (método recomendado para
diccionarios/JSON)
dto_curso = CursoIn.model_validate(raw_payload)

# Mapear a Entidad de Dominio
curso = Curso(
    nombre_curso=dto_curso.nombre_curso,
    estudiantes=[
        Estudiante(e.matricula, e.nombre, e.calificacion_promedio)
        for e in dto_curso.estudiantes
    ],
)

top = curso.estudiante_top
print(f"Curso creado: {curso.nombre_curso}")

if top is not None:
    print(
        f"Estudiante destacado (usando __lt__): {top.nombre}
({top.calificacion_promedio})"
    )
else:
    print("El curso no tiene estudiantes registrados.")

```

- Verificamos:

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundament
al_level/src/modulo4 (main)
$ poetry run mypy practica_teoría.py
Success: no issues found in 1 source file

```

```
juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
● $ git add .

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
● $ git commit -m "fix(modulo4) Se usa model_validate en raw_payload para resolver errores de mypy"
ruff.....Passed
ruff-format.....Passed
mypy.....Passed
[main d851279] fix(modulo4) Se usa model_validate en raw_payload para resolver errores de mypy
1 file changed, 9 insertions(+), 5 deletions(-)

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
● $ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 619 bytes | 619.00 KiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/juanrivera-A/capacitacion-python.git
578020a..d851279 main -> main
```

Verificación en GitHub:

- Creating checks

✓ **fix(modulo4) Se usa model_validate en raw_payload para resolver error... #3**



Summary ▾

Triggered via push 12 minutes ago

juanrivera-A pushed d851279 **main**

Status	Total duration	Artifacts
Success	<u>27s</u>	—

ci.yml

on: push

✓ **quality-checks** 22s



- Con esto confirmamos que:

1. La configuración del entorno de CI con Python 3.13 en GitHub Actions funciona correctamente.
2. Los análisis de Ruff, Ruff Formatter y Mypy se ejecutaron sobre todo el proyecto sin encontrar ninguna falla ni error de tipos.
3. Los candados de calidad locales (pre-commit) y remotos (GitHub Actions) están sincronizados al 100%.

Modulo 6. Librería estándar y Entrada/Salida (E/S).

En este módulo cubre los bloques fundamentales para construir scripts robustos de nivel profesional, automatizar tareas y manipular datos din dependiendo excesivamente de librerías de terceros.

1. Pathlib: Manejo moderno de rutas de archivos

Nos olvidamos de `os.path`. En python moderno, `pathlib` proporciona una interfaz orientada a objetos para interactuar con el sistema de archivos de forma agnóstica al sistema operativo (Windows/Linux/macOS).

Modulo 6. Libreria Estandar y E/S

Ingesta de CSV, métricas y exportación a JSON; logging con distintos niveles

Para cumplir con el Laboratorio del Modulo 6, integraremos una rutina de ingesta de CSVs, cálculo de métricas, exportación a JSON y logging estructurado por niveles (DEBUG, INFO, WARNING, ERROR).

1. Crear un CSV de prueba del archivo `data/ventas.csv`:

```
data > x ventas.csv > data
1 id,producto,cantidad,precio_unitario,categoria
2 1,Teclado Mecanico,5,1200.50,Perifericos
3 2,Mouse Gamer,0,450.00,Perifericos
4 3,Monitor 27,2,5500.00,Pantallas
5 4,Laptop Stand,-1,350.00,Accesorios
6 5,Cable HDMI,10,150.00,Accesorios
```

2. Creamos la lógica del laboratorio:

```
from typing import Any

from src.modulo6.config import DATA_DIR, LOGS_DIR, configurar_logger
from src.modulo6.parsers import guardar_json, leer_csv

logger = configurar_logger(LOGS_DIR)
```

```

def procesar_laboratorio() -> None:
    logger.info("Iniciando ejecución del laboratorio 06...")

    # 1. Manipulación segura de rutas con pathlin
    ruta_csv = DATA_DIR / "ventas.csv"
    ruta_json_metricas = DATA_DIR / "metricas_ventas.json"

    if not ruta_csv.exists():
        logger.error("El archivo de entrada no existe: %s", ruta_csv)
        raise FileNotFoundError(f"Archivo no encontrado: {ruta_csv}")

    # 2. Ingesta de CSV
    logger.debug("Leyendo archivo CSV desde %s", ruta_csv)
    registros = leer_csv(ruta_csv)
    logger.info("Se cargaron %d registros desde el CSV.",
len(registros))

    # 3. Procesamiento de métricas y logging con distintos niveles
    total_ventas = 0.0
    productos_procesados = 0
    anomalias_detectadas = 0

    for fila in registros:
        try:
            cantidad = int(fila.get("cantidad", 0))
            precio = float(fila.get("precio_unitario", 0.0))
            producto = fila.get("producto", "Desconocido")

            # Logging nivel WARNING para casos anómalos/no procesables
comercialmente
            if cantidad <= 0:
                logger.warning(
                    "Registro anómalo ignorado (cantidad <= 0) ->
Producto: %s | Cantidad: %d",
                    producto,
                    cantidad,
                )
                anomalias_detectadas += 1
                continue

            subtotal = cantidad * precio
            total_ventas += subtotal

```



```

        productos_procesados += 1

        logger.debug(
            "Procesado: %s | Cantidad: %d | Subtotal: $%.2f",
            producto,
            cantidad,
            subtotal
        )

    except ValueError as err:
        logger.error("Error al convertir tipos de datos en fila %s: %s", fila, err)

    ticket_promedio = (
        total_ventas / productos_procesados if productos_procesados > 0
    else 0.0
    )

    metricas: dict[str, Any] = {
        "total_registros_csv": len(registros),
        "registros_validos": productos_procesados,
        "registros_anomalos": anomalias_detectadas,
        "monto_total_ventas": round(total_ventas, 2),
        "ticket_promedio": round(ticket_promedio, 2),
    }

    logger.info("Métricas calculadas con éxito: %s", metricas)

    # 5. Exportación a JSON
    guardar_json(datos=metricas, ruta=ruta_json_metricas)
    logger.info("Exportación finalizada a %s", ruta_json_metricas)

if __name__ == "__main__":
    procesar_laboratorio()

```

3. Ejecución del laboratorio:

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ poetry run python -m src.modulo6.laboratorio
2026-09-21 20:56:54 [INFO] modulo_06: Iniciando ejecución del laboratorio 06...
2026-09-21 20:56:54 [INFO] modulo_06: Leyendo archivo CSV desde C:\Users\juan.rivera\Documents\Curso_Python\Fundamental_level\data\ventas.csv
2026-09-21 20:56:54 [INFO] modulo_06: Se cargaron 5 registros desde el CSV.
2026-09-21 20:56:54 [WARNING] modulo_06: Registro anómalo ignorado (cantidad <= 0) -> Producto: Mouse Gamer | Cantidad: 0
2026-09-21 20:56:54 [WARNING] modulo_06: Registro anómalo ignorado (cantidad <= 0) -> Producto: Laptop Stand | Cantidad: -1
2026-09-21 20:56:54 [INFO] modulo_06: Métricas calculadas con éxito: {'total_registros_csv': 5, 'registros_validos': 3, 'registros_anomalos': 2, 'monto_total_ventas': 18502.5, 'ticket_promedio': 6167.5}
2026-09-21 20:56:54 [INFO] modulo_06: Guardando archivo JSON en: C:\Users\juan.rivera\Documents\Curso_Python\Fundamental_level\data\metricas_ventas.json
2026-09-21 20:56:54 [INFO] modulo_06: Exportación finalizada a C:\Users\juan.rivera\Documents\Curso_Python\Fundamental_level\data\metricas_ventas.json

```

4. Ejecutamos las revisiones de calidad sobre el laboratorio:

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ poetry run mypy src
Success: no issues found in 21 source files

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ poetry run ruff check src/
All checks passed!

```

5. Realizamos el commit/push:

```

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git add .

juan.rivera@mxjuriagdell5400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   config/config.yaml
    new file:   data/auditoria.json
    new file:   data/metricas_ventas.json
    new file:   data/ventas.csv
    new file:   logs/app.log
    modified:   poetry.lock
    modified:   pyproject.toml
    new file:   src/__init__.py
    new file:   src/modulo6/__init__.py
    new file:   src/modulo6/config.py
    new file:   src/modulo6/laboratorio.py
    new file:   src/modulo6/logger.py
    new file:   src/modulo6/main.py
    new file:   src/modulo6/models.py
    new file:   src/modulo6/parsers.py
    new file:   src/modulo6/system.py

```

```

juan.rivera@mxjuriagdel15400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git commit -m "feat: Módulo. Librería estándar y E/S"
ruff.....Passed
ruff-format.....Passed
mypy.....Passed
[main 2db86b0] feat: Módulo. Librería estándar y E/S
16 files changed, 310 insertions(+), 4 deletions(-)
create mode 100644 config/config.yaml
create mode 100644 data/auditoria.json
create mode 100644 data/metricas_ventas.json
create mode 100644 data/ventas.csv
create mode 100644 logs/app.log
create mode 100644 src/__init__.py
create mode 100644 src/modulo6/__init__.py
create mode 100644 src/modulo6/config.py
create mode 100644 src/modulo6/laboratorio.py
create mode 100644 src/modulo6/logger.py
create mode 100644 src/modulo6/main.py
create mode 100644 src/modulo6/models.py
create mode 100644 src/modulo6/parsers.py
create mode 100644 src/modulo6/system.py

juan.rivera@mxjuriagdel15400 MINGW64 ~/Documents/Curso_Python/fundamental_level (main)
$ git push
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 8 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (21/21), 6.42 KiB | 1.07 MiB/s, done.
Total 21 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/juanrivera-A/capacitacion-python.git
d851279..2db86b0 main -> main

```

- Y verificamos en GitHub:

The screenshot shows the GitHub Actions interface for a workflow named "CI Quality Checks". The workflow is triggered by a push to the "main" branch and has a status of "Success". The total duration is 2m 36s. The workflow file is named "ci.yml" and is located in the root directory. The workflow consists of a single job named "quality-checks" which has a duration of 19s. The job is marked as successful with a green checkmark.

← CI Quality Checks

✓ feat: Módulo. Librería estándar y E/S #4 Re-run all jobs ...

Summary

All jobs

quality-checks

Run details

Usage

Workflow file

Triggered via push 15 hours ago

juanrivera-A pushed · 2db86b0 main

Status: Success

Total duration: 2m 36s

Artifacts: -

ci.yml
on: push

✓ quality-checks 19s

Modulo. Acceso a Datos y ORM

Laboratorio:

- Cliente httpx con reintentos/timeouts; descarga por streaming a disco
-
1. Definir los requerimientos y la arquitectura, el laboratorio constará de dos partes principales:
 - RobustHTTPClient (src/modulo7/robust_client.py):
 - Cliente sincrónico basado en httpx.Client .
 - Timeouts configurables (lectura, conexión).
 - Mecanismo de reintentos (Retry with Exponential Backoff): Reintenta peticiones ante fallos de red (httpx.RequestError) o errores del servidor (5xx).
 - Descarga por Streaming (download_file): Descarga archivos pesados en bloques (chunks) directamente a un archivo en disco sin cargar todo el contenido en memoria RAM.
 - Suite de pruebas (tests/test_robust_client.py):
 - Pruebas con pytest y Smocker para validar los reintentos ante fallos temporales (503 Service Unavailable).
 - Pruebas de descarga en streaming verificando la integridad del archivo descargado.

2. Crear el archivo Cliente HTTP Robusto (src/modulo7/robust_client.py):
 - Crear el archivo src/modulo7/robust_client.py

```
import time
from collections.abc import Callable
from pathlib import Path
from typing import Any

import httpx

class RobustHTTPClient:
    """Cliente HTTP con reintentos exponenciales, timeouts y soporte
    para streaming."""

    def __init__(
        self,
        base_url: str,
        timeout: float = 5.0,
        max_retries: int = 3,
        backoff_factor: float = 0.5,
    ) -> None:
        self.base_url = base_url.rstrip("/")
        self.timeout = httpx.Timeout(timeout)
        self.max_retries = max_retries
        self.backoff_factor = backoff_factor

    def _execute_with_retry[T](self, action: Callable[[], T]) -> T:
        """Ejecuta una acción con reintentos y backoff exponencial para
        errores 5xx y de red."""
        last_exception: Exception | None = None

        for attempt in range(1, self.max_retries + 1):
            try:
                return action()
            except (httpx.RequestError, httpx.HTTPStatusError) as exc:
                last_exception = exc

        # Si es un error HTTP de cliente (4xx), no reintentamos
        # (son errores permanente)
```

```

        if isinstance(exc, httpx.HTTPStatusError) and
exc.response.status_code < 500:
            raise exc

        if attempt == self.max_retries:
            break

        sleep_time = self.backoff_factor * (2 ** (attempt - 1))
        time.sleep(sleep_time)

    if last_exception:
        raise last_exception
    raise RuntimeError("Error inesperado e reintentos")

    def get_json(self, endpoint: str) -> dict[str, Any]:
        """Realiza una petición GET con reintentos y retorna el JSON
parsed."""
        url = f"{self.base_url}/{endpoint.lstrip('/')}"

        def _request() -> dict[str, Any]:
            with httpx.Client(timeout=self.timeout) as client:
                response = client.get(url)
                response.raise_for_status()
                return response.json() #type: ignore[no-any-return]

        return self._execute_with_retry(_request)

    def download_file(self, endpoint: str, destination_path: Path,
chunk_size: int = 8192) -> Path:
        """Descarga un archivo por streaming en bloques (chunks)
directamente a disco."""
        url = f"{self.base_url}/{endpoint.lstrip('/')}"

        def _download() -> Path:
            with (
                httpx.Client(timeout=self.timeout) as client,
                client.stream("GET", url) as response,
            ):
                response.raise_for_status()
                destination_path.parent.mkdir(parents=True,
exist_ok=True)
                with destination_path.open("wb") as file:

```

```

        for chunk in
response.iter_bytes(chunk_size=chunk_size):
            file.write(chunk)

    return destination_path

return self._execute_with_retry(_download)

```

3. Crear la suite de pruebas del laboratotio (tests/test_lab_modulo7.py)

Se validarán dos escenarios clave:

- Reintentos y resiliencia: Configurar Smocker para que responda primero con un error 503 Service Unavailable y luego de reintentar responda exitosamente con 200 ok
- Descarga en streaming a disco: Descargar un archivo binario/texto por streaming a un directorio temporal (tmp_path) y verificar que el contenido guardado en disco sea correcto.
- Crear el archivo tests/test_lab_modulo7.py

```

from pathlib import Path

import pytest

from src.modulo7.robust_client import RobustHTTPClient
from src.modulo7.smocker_client import SmockerClient

@pytest.fixture(autouse=True)
def setup_smocker() -> SmockerClient:
    """Fixture que reinicia el servidor Smocker antes de cada test."""
    client = SmockerClient()
    client.reset()
    return client

```

```

def test_get_json_with_retry_success(setup_smocker: SmockerClient) ->
None:
    # 1. Smocker responderá primero con 503 (1 vez) y luego con 200 OK
    # Smocker consume los mocks de forma secuencial
    setup_smocker.add_mock(
        [
            {
                "request": {"method": "GET", "path": "/api/v1/retry-
test"},
                "response": {
                    "status": 503,
                    "body": '{"error": "Service Temporarily
Unavailable"}',
                },
            },
            {
                "request": {"method": "GET", "path": "/api/v1/retry-
test"},
                "response": {
                    "status": 200,
                    "headers": {"Content-Type": "application/json"},
                    "body": '{"status": "recovered", "attempt": 2}',
                },
            },
        ]
    )

    # 2. Cliente con backoff factor muy pequeño para no ralentizar los
tests
    client = RobustHTTPClient(
        base_url="http://localhost:8080",
        max_retries=3,
        backoff_factor=0.01,
    )

    # 3. La petición debe tener éxito en el segundo intento
    data = client.get_json("/api/v1/retry-test")
    assert data["status"] == "recovered"
    assert data["attempt"] == 2

```



```

def test_download_file_streaming(setup_smocker: SmockerClient,
tmp_path: Path) -> None:
    expected_content = "Contenido de archivo pesado en bloques para
streaming."

    # 1. Registrar mock del archivo a descargar
    setup_smocker.add_mock(
        [
            {
                "request": {"method": "GET", "path":
"/files/report.txt"},
                "response": {
                    "status": 200,
                    "headers": {"Content-Type": "text/plain"},
                    "body": expected_content,
                },
            }
        ]
    )

    # 2. Configurar cliente y ruta de destino usando el fixture
tmp_path de pytest
    client = RobustHTTPClient(base_url="http://localhost:8080")
    destination_file = tmp_path / "downloads" / "downloaded_report.txt"

    # 3. Descargar por streaming
    saved_path = client.download_file("/files/report.txt",
destination_file)

    # 4. Validar que el archivo existe en disco y su contenido es
exacto
    assert saved_path.exists()
    assert saved_path.read_text(encoding="utf-8") == expected_content

```

4. Ejecutar las pruebas del laboratorio y las herramientas de calidad:

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/fundamental_level (main)
$ poetry run pytest tests/test_lab_modulo7.py -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:/Users/juan.rivera/AppData/Local/pypoetry\Cache/virtualenvs/fundamental-level-Pz1AG0Hn-py3.13\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:/Users/juan.rivera/Documents/Curso_Python/fundamental_level
configfile: pyproject.toml
plugins: anyio-4.15.1
collected 2 items

tests/test_lab_modulo7.py::test_get_json_with_retry_success PASSED [ 50%]
tests/test_lab_modulo7.py::test_download_file_streaming PASSED [100%]

===== 2 passed in 2.68s =====

```

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/fundamental_level (main)
$ poetry run ruff check .
All checks passed!

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/fundamental_level (main)
$ poetry run mypy src
Success: no issues found in 29 source files

```

5. Realizar el commit y push:

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/fundamental_level (main)
$ git commit -m "feat: Módulo. HTTP y consumo de APIs(Smocker)"
ruff.....(no files to check)Skipped
ruff-format.....(no files to check)Skipped
mypy.....(no files to check)Skipped
[main 72b0f6e] feat: Módulo. HTTP y consumo de APIs(Smocker)
1 file changed, 4 insertions(+), 1 deletion(-)

```

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/fundamental_level (main)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 369 bytes | 369.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/juanrivera-A/capacitacion-python.git
2a80c61..72b0f6e main -> main

```

5 workflow runs
Workflow ▾
Event ▾
Status ▾
Branch ▾
Actor ▾

feat: Módulo. HTTP y consumo de APIs(Smocker)
main
🕒 2 minutes ago
⌵

CI Quality Checks #6: Commit 72b0f6e pushed by juanrivera-A
🕒 24s